這次我不再重新搜索——前面已經累積了足夠的素材。我直接用我們建立的所有知識做一次完整的、從物理振動到使用者意圖的全棧設計，用助聽器場景中的六個人來驗證每一層。

完整場景：李伯伯的一天

設定：72 歲的李伯伯戴著新一代智慧助聽器，從早上出門到晚上看電視。助聽器內建了語義 ISA + Harness 系統。以下是系統在他一天中面對的具體時刻。

08:30 — 在家裡跟老伴說話（安靜，偶爾有冰箱嗡嗡聲） **12:15** — 在菜市場買菜（極嘈雜，多人同時說話） **15:00** — 在公園散步，孫子在旁邊跑（風聲、鳥叫、兒童笑聲） **19:30** — 看電視新聞，老伴同時在旁邊跟他說話（電視聲 + 對話聲重疊）

我用**12:15 菜市場**這個最具挑戰性的時刻作為核心場景，展開完整的架構。

第一部分：完整的 Semantic IR——從物理層到意圖層

IR 的七層結構

我把 IR 設計為七層，每一層都有明確的型別定義和指令定義。**每一層的指令分為 Primitive（不可再分解的原子操作）和 Composite（由 Primitive 組合而成的複合操作）**。

```
; ═══════════════════════════════════════════════════════
; Acoustic Semantic IR (ASIR) - Version 0.1 Draft
; 完整的七層中間表示，從物理振動到使用者意圖
; ═══════════════════════════════════════════════════════


; ═══ 第一層：物理感測層 (Physical Sensing Layer) ═══════════
; 這一層完全確定性，跟傳統 DSP ISA 完全對應
; 所有指令都是 Primitive，由硬體直接實現

TYPE RawSignal = Tensor<float32>[channels, samples]
TYPE SampleRate = int  ; Hz
TYPE BitDepth = int
TYPE MicGeometry = Array<(float, float, float)>  ; 每個麥克風的 3D 座標

; --- Primitive Instructions ---
PRIM sample_audio(mics: MicGeometry, duration: float, sr: SampleRate)
  -> RawSignal
  ; 硬體直接實現，完全確定性
  ; 可靠性：100%（硬體保證）
  ; 類比：CPU 的 LOAD 指令

PRIM read_imu() -> AccelerometerData
  ; 讀取慣性測量單元（頭部運動）
  ; 用於波束追蹤
```

```
; ═══ 第二層：信號處理層 (Signal Processing Layer) ═══════════
; 仍然完全確定性，傳統 DSP
; Primitive 由數學定義保證正確

TYPE Spectrum = Tensor<complex64>[channels, freq_bins]
TYPE FilterCoeffs = Tensor<float32>[order]
TYPE BeamPattern = Tensor<float32>[azimuth_bins, elevation_bins]

; --- Primitive Instructions ---
PRIM fft(signal: RawSignal, window: WindowType, size: int) -> Spectrum
  ; 完全確定性，數學保證正確
  ; 可靠性：100%（數值精度內）

PRIM apply_filter(signal: RawSignal, coeffs: FilterCoeffs) -> RawSignal
  ; 確定性 FIR/IIR 濾波

PRIM estimate_noise_psd(signal: RawSignal, method: NoiseEstMethod)
  -> Spectrum
  ; MMSE 或 minimum statistics 噪音估計
  ; 確定性演算法

PRIM beamform(signals: RawSignal, geometry: MicGeometry,
              direction: (float, float)) -> RawSignal
  ; 固定方向波束成形，確定性

; --- Composite Instructions ---
COMP spectral_subtract(signal: RawSignal, noise: Spectrum,
                       alpha: float) -> RawSignal
  = fft(signal) |> subtract_magnitude(noise, alpha) |> ifft()
  ; 由三個 primitive 組合，仍然完全確定性


; ═══ 第三層：聲學特徵層 (Acoustic Feature Layer) ═══════════
; 確定性，傳統信號處理
; 但開始出現「什麼特徵有意義」的設計選擇

TYPE AcousticFeatures = {
  mfcc: Tensor<float32>[n_frames, n_coeffs],
  pitch: Tensor<float32>[n_frames],        ; 基頻軌跡
  energy: Tensor<float32>[n_frames],       ; 能量包絡
  spectral_tilt: Tensor<float32>[n_frames],
  snr_estimate: float,
  rt60_estimate: float                      ; 混響時間估計
}

; --- Primitive Instructions ---
PRIM extract_mfcc(signal: RawSignal, n_coeffs: int)
  -> Tensor<float32>[n_frames, n_coeffs]

PRIM estimate_pitch(signal: RawSignal, method: PitchMethod)
  -> Tensor<float32>[n_frames]

PRIM estimate_rt60(signal: RawSignal) -> float
```

```
  ; 從信號中估計混響時間

; --- Composite Instructions ---
COMP extract_full_features(signal: RawSignal) -> AcousticFeatures
  = {
    mfcc: extract_mfcc(signal, 13),
    pitch: estimate_pitch(signal, "YIN"),
    energy: compute_energy(signal),
    spectral_tilt: compute_tilt(fft(signal)),
    snr_estimate: estimate_snr(signal),
    rt60_estimate: estimate_rt60(signal)
  }


; ═══════════════════════════════════════════════════════════
; ▲▲▲ 以上三層：完全確定性，傳統 DSP ISA 覆蓋 ▲▲▲
; ═══════════════ SEMANTIC BOUNDARY ═══════════════════
; ▼▼▼ 以下四層：進入語義空間，需要 LLM ▼▼▼
; ═══════════════════════════════════════════════════════


; ═══ 第四層：感知描述層 (Perceptual Description Layer) ═══
; ★ 這是 Shannon 切割的修復點 ★
; 從數值特徵翻譯成人類可理解的語義描述
; 這一層的指令是 LLM 驅動的，可靠性是統計性的

TYPE PerceptualDescription = {
  noise: {
    perceived_level: Enum<silent|faint|noticeable|loud|overwhelming>,
    sources: List<{type: string, direction: string, temporal: string}>,
    ; 例如：{type: "多人交談聲", direction: "四周環繞", temporal: "持續"}
    ;       {type: "金屬碰撞", direction: "右前方", temporal: "偶發"}
  },
  speech: {
    n_speakers: int,
    target_speaker: {direction: string, gender: string,
                     estimated_distance: string},
    intelligibility: Enum<clear|slightly_masked|heavily_masked|inaudible>
  },
  environment: {
    type: string,           ; "室內菜市場"
    reverberation: string,  ; "中等混響，硬地面反射明顯"
    acoustic_character: string  ; "嘈雜、動態、多聲源"
  },
  confidence: float[0,1]
}

; --- Primitive Instructions ---
; ★ 這些是語義 ISA 的核心——它們是不可再分解的語義原子操作

PRIM describe_noise(features: AcousticFeatures)
  -> PerceptualDescription.noise
  BACKEND: LLM
```

```
  RELIABILITY: source_identification_accuracy >= 0.80
  FAILURE_MODE: unknown_source -> returns {type: "unidentified", ...}
  ; 這條指令做的事：把 SNR 數字、頻譜形狀等數值特徵
  ; 翻譯成「多人交談聲，四周環繞，持續」這種人類感知描述
  ; ★ 這個翻譯在傳統演算法中不可能——因為「多人交談聲」
  ;   不是一個可以從數值特徵精確計算出來的東西

PRIM describe_speech(features: AcousticFeatures)
  -> PerceptualDescription.speech
  BACKEND: LLM
  RELIABILITY: n_speakers_accuracy >= 0.85 (for 1-4 speakers)
  FAILURE_MODE: >4 speakers -> returns n_speakers = "many"

PRIM describe_environment(features: AcousticFeatures)
  -> PerceptualDescription.environment
  BACKEND: LLM
  RELIABILITY: environment_type_accuracy >= 0.75
  FAILURE_MODE: novel_environment -> returns confidence < 0.5

; --- Composite Instructions ---
COMP full_perceptual_description(features: AcousticFeatures)
  -> PerceptualDescription
  = {
    noise: describe_noise(features),
    speech: describe_speech(features),
    environment: describe_environment(features),
    confidence: min(noise.confidence, speech.confidence, env.confidence)
  }
  ; 組合三個 primitive，confidence 取最低值（保守策略）


; ═══ 第五層：場景理解層 (Scene Understanding Layer) ═══════
; 從感知描述推理出更高層的場景語義
; 這一層需要跨維度的推理——不只是列舉事實，而是「理解」場景

TYPE SceneUnderstanding = {
  situation: string,
  ; "李伯伯在菜市場的魚攤前，攤販正在跟他說話，
  ;  旁邊有其他顧客在交談，背景有切魚和冰塊碰撞的聲音"

  acoustic_challenges: List<{
    challenge: string,
    severity: Enum<mild|moderate|severe>,
    physical_cause: string
  }>,
  ; [{challenge: "目標語音被多人交談聲遮蔽",
  ;   severity: severe,
  ;   physical_cause: "多個語音源的頻譜重疊在 300-3000Hz"}]

  preservation_notes: List<string>,
  ; ["環境聲中的金屬碰撞聲可幫助李伯伯定位攤位，不應完全去除"]
  ; ★ 這種判斷在傳統演算法中根本不存在——
  ;   「哪些聲音對使用者有導航價值」是語義層的判斷
```

```
  confidence: float[0,1]
}

; --- Primitive Instructions ---
PRIM reason_about_scene(percept: PerceptualDescription,
                        user_context: UserProfile)
  -> SceneUnderstanding
  BACKEND: LLM (requires strong reasoning capability)
  RELIABILITY:
    situation_relevance >= 0.80 (judged by LLM-as-judge)
    challenge_identification >= 0.75
  FAILURE_MODE:
    low_confidence -> returns only acoustic_challenges, skips situation
  ; ★ 這是一條 Primitive，因為「理解場景」不能被分解為更小的
  ;   語義操作——它需要同時考慮所有感知維度並做跨維度推理

; --- Composite Instructions ---
COMP scene_with_history(percept: PerceptualDescription,
                        user_context: UserProfile,
                        recent_scenes: List<SceneUnderstanding>)
  -> SceneUnderstanding
  = reason_about_scene(percept, user_context)
    |> merge_with_history(recent_scenes)
    |> resolve_contradictions()
  ; 如果 30 秒前場景是「菜市場魚攤」，現在突然變成「安靜辦公室」，
  ; resolve_contradictions 會質疑新判斷（可能是暫時的安靜，不是場景切換）


; ═══ 第六層：策略生成層 (Strategy Generation Layer) ═══════
; ★ 這是目前整個研究領域最大的空缺 ★
; 從場景理解生成具體的信號處理策略
; 這一層是「語意-物理介面」的核心

TYPE ProcessingStrategy = {
  beamforming: {
    target_direction: (float, float),  ; 方位角、仰角
    beam_width: float,                 ; 波束寬度（度）
    null_directions: List<(float, float)>,  ; 零點方向
    adaptation_rate: Enum<slow|medium|fast>
  },
  noise_reduction: {
    method: Enum<spectral_subtraction|wiener|dnn_masking>,
    aggressiveness: float[0,1],
    ; 0 = 最保守（保留更多環境聲），1 = 最激進（最大降噪）
    preserve_bands: List<(float, float)>,
    ; 需要保留的頻段（例如保留環境定位線索的頻段）
  },
  gain: {
    frequency_response: Tensor<float32>[freq_bins],
    compression_ratio: float,
    attack_time: float,
    release_time: float
```

```
  },
  mixing: {
    direct_to_processed_ratio: float,
    ; 0 = 完全處理後的聲音，1 = 完全原始聲音
    ; 允許使用者「聽到一些真實環境」
  },
  confidence: float[0,1],
  reasoning: string
  ; "因為李伯伯在嘈雜菜市場且面前有人在說話，
  ;  我將波束對準正前方 ±20°，降噪設為中等攻擊性 (0.6)，
  ;  但保留 200-800Hz 的環境聲以幫助空間感知。
  ;  根據李伯伯的偏好，不完全去除背景人聲，
  ;  因為他希望知道周圍有人在活動。"
}

; --- Primitive Instructions ---
PRIM generate_beamforming_params(scene: SceneUnderstanding)
  -> ProcessingStrategy.beamforming
  BACKEND: LLM + physics_constraints
  RELIABILITY: target_direction_error <= 15° in 90% of cases
  CONSTRAINT: beam_width >= 20° (物理約束：麥克風陣列的最小波束寬度)
  ; ★ Primitive，因為波束參數的生成需要同時考慮
  ;   語義（「目標說話者在正前方」）和物理（陣列幾何約束）

PRIM generate_noise_reduction_params(scene: SceneUnderstanding,
                                     user_prefs: UserPreferences)
  -> ProcessingStrategy.noise_reduction
  BACKEND: LLM
  RELIABILITY: 策略應用後 PESQ 提升 >= 0.3 in 80% of cases
  CONSTRAINT: aggressiveness <= user_prefs.max_aggressiveness
  ; ★ Primitive，因為降噪參數需要權衡使用者偏好和場景需求

PRIM generate_gain_params(scene: SceneUnderstanding,
                          audiogram: UserAudiogram)
  -> ProcessingStrategy.gain
  BACKEND: deterministic (NAL-NL2 or DSL v5 fitting formula)
  RELIABILITY: 100% (確定性演算法)
  ; 注意：這條 Primitive 不用 LLM——增益計算有成熟的聽力學公式
  ; 但它需要場景理解的輸出來決定壓縮比等動態參數

; --- Composite Instructions ---
COMP generate_full_strategy(scene: SceneUnderstanding,
                            user: UserProfile)
  -> ProcessingStrategy
  = {
    beamforming: generate_beamforming_params(scene),
    noise_reduction: generate_noise_reduction_params(scene, user.prefs),
    gain: generate_gain_params(scene, user.audiogram),
    mixing: determine_mixing_ratio(scene, user.prefs),
    confidence: ...,
    reasoning: synthesize_reasoning(...)
  }
```

```
; ★★★ 關鍵的 Composite：語意-物理翻譯 ★★★
COMP strategy_to_dsp_params(strategy: ProcessingStrategy)
  -> DSPParameterSet
  = {
    ; 把語義層的策略翻譯成第二層 DSP 的具體參數
    filter_coeffs: compute_filter(strategy.gain.frequency_response),
    beam_weights: compute_beam_weights(strategy.beamforming, mic_geometry),
    noise_mask: compute_noise_mask(strategy.noise_reduction),
    ...
  }
  BACKEND: deterministic (純數學計算)
  ; ★ 這個 Composite 就是「語意-物理介面」的具體實現
  ; 它的輸入是語義空間的（ProcessingStrategy）
  ; 它的輸出是語法空間的（DSPParameterSet）
  ; 翻譯本身是確定性的——一旦策略確定，DSP 參數就確定


; ═══ 第七層：意圖與偏好層 (Intent & Preference Layer) ═══
; 最頂層——直接面對使用者的語義世界

TYPE UserIntent = {
  primary_goal: string,
  ; "我想聽清楚面前賣魚的在說什麼價錢"

  secondary_goals: List<string>,
  ; ["我想知道周圍大概多擠", "不要讓我覺得跟環境隔離"]

  constraints: List<string>,
  ; ["不要有那種機器處理過的感覺", "寧可聽不太清楚也不要失真"]

  context: string
  ; "我每週三都來這個菜市場，通常在魚攤待 10 分鐘"
}

TYPE UserPreferences = {
  noise_tolerance: Enum<low|medium|high>,
  ; 從歷史互動中學到的：李伯伯 = medium（能接受一些噪音）

  processing_preference: Enum<natural|balanced|maximal_clarity>,
  ; 李伯伯 = natural（偏好自然聲）

  environment_awareness: Enum<minimal|moderate|full>,
  ; 李伯伯 = moderate（想感知環境但不需要聽到所有細節）

  known_difficult_situations: List<{situation: string,
                                    preferred_strategy: string}>,
  ; [{situation: "菜市場", preferred_strategy: "增強正前方，保留環境感"},
  ;  {situation: "看電視", preferred_strategy: "增強電視方向，降低其他"}]

  last_updated: datetime
}

; --- Primitive Instructions ---
```

```
PRIM parse_intent(user_input: string) -> UserIntent
  BACKEND: LLM
  RELIABILITY: primary_goal_extraction >= 0.90
  ; 使用者可能不會每次都說話——可能只是按了一個按鈕
  ; 這時候 intent 從歷史偏好和當前場景推斷

PRIM update_preferences(current_prefs: UserPreferences,
                        feedback: UserFeedback,
                        current_scene: SceneUnderstanding)
  -> UserPreferences
  BACKEND: LLM
  RELIABILITY: preference_consistency >= 0.85
  ; ★ Primitive，因為從單次回饋推斷長期偏好變化是一個
  ;   不可分解的語義推理操作

; --- Composite Instructions ---
COMP intent_aware_strategy(audio: RawSignal,
                           user: UserProfile)
  -> (ProcessingStrategy, ExplanationForUser)
  = {
    features = extract_full_features(audio)          ; 第三層
    percept = full_perceptual_description(features)  ; 第四層
    scene = scene_with_history(percept, user, ...)   ; 第五層
    strategy = generate_full_strategy(scene, user)   ; 第六層
    dsp_params = strategy_to_dsp_params(strategy)    ; 第六→二層翻譯
    explanation = generate_user_explanation(strategy) ; 給使用者的解釋
    RETURN (strategy, explanation)
  }
  ; ★★★ 這個 Composite 就是整個管線的最高層入口 ★★★
  ; 它串接了所有七層，從 RawSignal 一路到 ProcessingStrategy
```

Primitive vs. Composite 的完整清單

```
第一層（物理）      Primitive: sample_audio, read_imu
                    Composite: (無)

第二層（信號處理）  Primitive: fft, apply_filter, estimate_noise_psd, beamform
                    Composite: spectral_subtract

第三層（特徵）      Primitive: extract_mfcc, estimate_pitch, estimate_rt60
                    Composite: extract_full_features

────── SEMANTIC BOUNDARY ──────

第四層（感知描述）  Primitive: describe_noise, describe_speech, describe_environment
                    Composite: full_perceptual_description

第五層（場景理解）  Primitive: reason_about_scene
                    Composite: scene_with_history

第六層（策略生成）  Primitive: generate_beamforming_params,
```

```
                                generate_noise_reduction_params,
                                generate_gain_params
                        Composite: generate_full_strategy, strategy_to_dsp_params

第七層（意圖偏好） Primitive: parse_intent, update_preferences
                        Composite: intent_aware_strategy
```

注意 Primitive 和 Composite 的判定標準：一個操作是 Primitive，如果它不能被有意義地分解為更小的、可獨立測試的語義操作。`describe_noise` 是 Primitive，因為「從聲學特徵推斷噪音的感知描述」是一個不可分割的語義跳躍——你不能把它拆成「先判斷噪音大小」再「判斷噪音類型」再「判斷噪音方向」然後組合，因為這三者之間有強耦合（噪音大小影響你對類型的判斷，方向影響你對大小的感知）。

相反，`full_perceptual_description` 是 Composite，因為噪音描述、語音描述、環境描述可以被獨立測試和替換——你可以改進 describe_noise 而不影響 describe_speech。

第二部分：六個人圍著這份 IR 的反應

場景：設計會議，白板上投影著上面的 IR 草案

陳教授（DSP 老將，看著第一到第三層）

> 「前三層我完全看得懂——這就是我做了四十年的東西。FFT、波束成形、MFCC——每一步我都可以手寫數學證明。它們是 Primitive，因為它們有精確的數學定義。」
>
> 「但你在第二層的 spectral_subtract 定義為 Composite 而非 Primitive——我同意。因為它確實可以被分解為 FFT + 減法 + IFFT，而且你可以把減法替換成 Wiener 濾波或其他方法而不影響前後步驟。**可組合性就是 Composite 和 Primitive 的分界。**」
>
> 「我注意到你在第三層的 estimate_rt60 也標記為 Primitive。嚴格來說，RT60 估計可以被分解為更小的步驟（計算能量衰減曲線 → 擬合斜率 → 換算為秒數）。但我理解你為什麼把它當 Primitive——因為在語義 ISA 的層次上，我們不在乎**怎麼**估計 RT60，我們只在乎它**返回一個 float**。**ISA 的 Primitive 不是說它在實現上不可分解，而是說在這一層的抽象上它是不可分解的。** 就像 x86 的 MUL 指令在微架構裡可能被分解為多個 micro-ops，但在 ISA 層面它是一條 Primitive。」

王博士後（ML 研究者，盯著第三到第四層之間的 Semantic Boundary）

> 「那道 Semantic Boundary 讓我很不安。你在說第三層以上全部是確定性的，第四層以下開始是統計性的。但——」
>
> 「我的 DNN 語音增強模型跨越了這道邊界。它的輸入是第一層的 RawSignal，輸出直接是增強後的 RawSignal——它跳過了所有中間層。在你的 IR 裡，它放在哪裡？」
>
> 「我想你的回答會是：我的 DNN 是一個不透明的 Composite——它在內部隱含地做了第二層到第四層的所有事情，但它不把中間結果暴露出來。而你的 IR 的價值在於它**強制把中間結果暴露為有型別的、可檢查的值**。如果我的 DNN 直接輸出增強語音，沒有人知道它在內部是怎麼判斷場景的。但如果用你的 IR，scene 變數在第五層被顯式定義為 SceneUnderstanding 型別——任何人都可以檢查這個中間值是否合理。」
>
> 「**這就是 IR 的價值——不是效能，而是可觀測性和可除錯性。** LLVM IR 的價值也不是它跑得比直接編譯更快——而是它讓你可以在中間層做分析和優化。」

Priya（系統架構師，看著 Harness 的角色）

> 「讓我把 Harness 在這份 IR 中的角色畫出來。你們看——」

```
                    ┌──────────────────────────────┐
                    │          HARNESS             │
                    │                              │
   ┌────────────────┤  Semantic Linker:            │
   │                │  • 第四層輸出→第五層輸入的    │
   │                │    格式適配和衝突檢測         │
   │                │  • 第六層輸出→第二層輸入的    │
   │                │    語意-物理翻譯              │
   │                │                              │
   │   ┌────────────┤  Semantic Runtime:           │
   │   │            │  • Context window 管理       │
   │   │            │  • Tool dispatch             │
   │   │            │    (第四層呼叫 LLM,           │
   │   │            │     第二層呼叫 DSP 硬體)      │
   │   │            │  • Error recovery            │
   │   │            │    (LLM 幻覺偵測 → retry)     │
   │   │            │                              │
   │   │   ┌────────┤  Semantic Scheduler:         │
   │   │   │        │  • 第四層用哪個 LLM？         │
   │   │   │        │    (便宜的 mini 還是貴的 4o?) │
   │   │   │        │  • Token 預算分配             │
   │   │   │        │  • 延遲約束                   │
   │   │   │        │    (助聽器需要 <50ms)         │
   │   │   │        │                              │
   │   │   │   ┌────┤  Persistent Store:           │
   │   │   │   │    │  • UserPreferences 持久化     │
   │   │   │   │    │  • 場景歷史 (last N scenes)   │
   │   │   │   │    │  • 效能 log (用於 JIT 再優化) │
   │   │   │   │    └──────────────────────────────┘
   │   │   │   │
   ▼   ▼   ▼   ▼
 IR 的每一層都透過 Harness 的不同子系統互動
```

> 「注意 Harness 的四個子系統分別對應 LLVM 生態中的不同角色：」
>
> 「**Semantic Linker** 對應 lld（LLVM Linker）——它負責把各層的模組連結成可運作的管線。當第四層的 describe_noise 輸出一個 PerceptualDescription.noise，而第五層的 reason_about_scene 期望接收一個完整的 PerceptualDescription——Linker 負責組裝。」
>
> 「**Semantic Runtime** 對應 compiler-rt + libc——它提供運行時基礎服務。Context window 管理就像 malloc/free——分配和回收有限的 token 空間。Error recovery 就像 exception handling——當 LLM 返回幻覺時 catch 住並 retry。」
>
> 「**Semantic Scheduler** 對應 OS 的 process scheduler——它決定每個 LLM 呼叫用什麼模型、分配多少資源。在助聽器場景中，這特別重要——你只有 50ms 的延遲預算，不可能每一步都呼叫 GPT-4o。Scheduler 需要決定：第四層的 describe_noise 可以用便宜的小模型（延遲低），但第五層的 reason_about_scene 需要大模型（推理能力強）。」

> 「**Persistent Store** 對應 OS 的 file system——它持久化跨 session 的狀態。李伯伯的偏好、常去的場景、歷史表現——這些都存在 Persistent Store 裡，每次新 session 開始時 Harness 載入它們。」

張博士生（你的角色——看著第六層）

> 「我一直盯著第六層看，因為這是我最在乎的地方。」
>
> 「你們注意到了嗎——第六層的三個 Primitive 指令中，generate_gain_params 的 BACKEND 是 deterministic，不是 LLM。因為增益計算有成熟的聽力學公式（NAL-NL2、DSL v5）——這是幾十年的聽力學研究已經解決的問題。**不是所有在語義邊界以上的操作都需要 LLM。有些操作雖然需要語義輸入（場景理解的結果），但執行本身是確定性的。**」
>
> 「這就是 Primitive 的 BACKEND 標記的重要性：」

```
第六層的三個 Primitive：
  generate_beamforming_params  → BACKEND: LLM + physics_constraints
  generate_noise_reduction_params → BACKEND: LLM
  generate_gain_params         → BACKEND: deterministic
```

> 「BACKEND 可以是 LLM、deterministic、或混合。**一個 Primitive 的語義定義不依賴於它的 BACKEND——就像 x86 ISA 的 ADD 指令不關心底下是用 ripple-carry adder 還是 carry-lookahead adder 實現的。**」
>
> 「但更深的問題是 strategy_to_dsp_params 這個 Composite。它是整個 IR 中最奇特的一個——它的**輸入**在語義空間（ProcessingStrategy，包含自然語言描述如 '增強正前方 ±20°'），它的**輸出**在物理空間（DSPParameterSet，包含精確的濾波器係數）。它跨越了 Semantic Boundary——是一個**下行翻譯器**。」
>
> 「而且它是 deterministic 的！一旦 ProcessingStrategy 中的所有數值參數確定了（beam_width = 40°、aggressiveness = 0.6），翻譯成 DSP 參數就是純數學。**語義到物理的翻譯本身不需要 LLM——LLM 的作用在於生成 ProcessingStrategy，翻譯過程是確定性的。**」
>
> 「這對我的 cross-sensor 研究直接相關：如果我能定義一個 CrossSensorConversionStrategy 型別（包含 '頻段權重' '相位補償方法' 等語義描述），然後有一個確定性的 strategy_to_conversion_params 把它翻譯成 DT 模型的具體參數——那我的貢獻就是：(1) 定義 Strategy 型別的結構，(2) 設計 generate_conversion_strategy 這個 Primitive，(3) 證明整個管線在 GEPA 優化下的可靠性。」

產品經理（看著第七層）

> 「我最關心第七層。讓我用李伯伯在菜市場的例子走一遍。」
>
> 「李伯伯不會跟助聽器說 '請把波束對準正前方 ±20°'。他會做的事情是：」
>
> 「**情境 A**：他面向攤販，什麼都沒做——系統需要自動偵測他在跟攤販說話。」
>
> 「**情境 B**：他按了一下助聽器上的按鈕——表示 '我聽不清楚'。系統需要從這個二元回饋中推斷：是噪音太大？是波束方向錯了？是攤販聲音太小？」
>
> 「**情境 C**：半年後，系統應該已經學會 '李伯伯每週三去菜市場，在菜市場他喜歡中等降噪因為他想感知環境'——這個偏好不是他明說的，是系統從他的按鈕回饋模式中推斷的。」
>
> 「在你的 IR 裡，情境 A 對應 parse_intent——但輸入不是文字，而是 '使用者沒有給任何輸入'。Primitive 需要能處理**缺少明確輸入**的情況——從場景和歷史推斷意圖。」

> 「情境 B 對應 update_preferences——從一次按鈕回饋推斷長期偏好的微調。這是一個極其困難的推理——一個二元信號（滿意/不滿意）加上當前場景的完整 context，要推斷出偏好向量的哪個維度需要調整。」
>
> 「情境 C 涉及 Harness 的 Persistent Store——跨越數月的偏好累積。而且偏好會變——李伯伯的聽力可能在退化，他半年前覺得 OK 的降噪強度現在可能不夠了。update_preferences 需要偵測偏好的漂移（drift），不只是累積。」

陳教授（最後發言——看著整體架構）

> 「我現在看到了一件非常深刻的事情。」
>
> 「你的 IR 有七層。前三層完全確定性，後四層涉及語義。但它不是簡單的 '上面是語義、下面是物理' 的兩段結構——因為**第六層的 strategy_to_dsp_params 建立了一條回路**：」

```
第七層  意圖       ──→ 第六層  策略 ──→ strategy_to_dsp_params
                                                  │
                                                  ↓
第一層  物理感測 ←── 第二層  DSP ←──────── DSPParameterSet
      │
      ↓
第三層  特徵 ──→ 第四層  感知描述 ──→ 第五層  場景理解 ──→ 第六層  ...
```

> 「**這是一個迴路，不是一條直線。** 物理層的信號被分析為特徵、被描述為感知、被理解為場景、被生成為策略——然後策略被翻譯回物理層的 DSP 參數，改變了信號的處理方式。處理後的信號在下一個時間步又被感測——迴路閉合了。」
>
> 「在 LLVM 的架構中，編譯是一次性的——源碼 → IR → 機器碼，然後執行。但你的 IR 描述的是一個**持續運行的控制迴路**——每隔幾十毫秒，迴路轉一圈。這更像是控制理論中的閉環系統，而不是編譯器的單向管線。」
>
> 「所以嚴格來說，你的 IR 不只是 LLVM IR 的類比——**它是 LLVM IR 加上一個控制迴路的語義。** 傳統 LLVM IR 不需要表達 '這段程式碼的輸出會影響它下一次執行的輸入'，但你的 Acoustic Semantic IR 必須表達這一點——因為助聽器是一個即時控制系統，不是一個 batch 處理管線。」
>
> 「這意味著你的 IR 需要 LLVM IR 沒有的東西：**時態語義（temporal semantics）**——不只是 '這一步的輸出是什麼型別'，還有 '這一步的輸出多久之後會影響哪一步的輸入' '如果下一圈的感測結果跟這一圈矛盾，哪一圈的判斷優先'。」
>
> **「這是你的 IR 跟 LLVM IR 最本質的區別：LLVM IR 描述的是一次計算的結構。你的 IR 描述的是一個持續運行的、跨越語法和語義兩個空間的控制迴路的結構。」**

第三部分：你可能忽略的關鍵事項

基於上面的完整展開，以下是可能被忽略但非常重要的要素：

1. 延遲約束改變了整個 IR 的語義

助聽器的延遲預算是 30-50ms。這意味著你不可能每一幀都跑完七層。實際的執行模式是：

```
快速通道（每幀，<10ms）：
  第一層 → 第二層 → 用上一次的 DSP 參數處理 → 輸出

慢速通道（每 500ms-2s）：
  第三層 → 第四層 → 第五層 → 更新場景判斷

極慢通道（使用者主動觸發或每 30s）：
  第六層 → 更新處理策略

背景通道（非即時）：
  第七層 → 偏好更新
```

IR 需要表達這種多速率的執行模式。 不是所有指令在每一幀都執行——有些指令在「快車道」上跑（確定性 DSP），有些在「慢車道」上跑（LLM 推理）。Harness 的 Scheduler 管理不同車道的同步和資料交換。

LLVM IR 不需要表達這個——因為傳統程式不是即時控制系統。但你的 IR 需要。

2. 向下回饋路徑（Feedback Path）

第六層的 strategy_to_dsp_params 是一條**向下的路徑**——從語義空間回到物理空間。但還有其他向下回饋：

— 如果第五層的 scene_understanding 改變了（從「安靜辦公室」變成「菜市場」），需要通知第三層**改變特徵提取策略**（在安靜環境下不需要提取噪音特徵，在嘈雜環境下需要）。 — 如果第七層的 user_preferences 更新了（使用者按了「太吵」的按鈕），需要**即時**修改第六層的策略，不等到下一個慢速通道週期。

IR 需要支援「中斷」語義——某些事件可以打斷正常的多速率執行流程。 這類比 CPU 的 interrupt handling，但在語義空間中。

3. Primitive 的可靠性不是固定的——它依賴上下文

我在 IR 中寫了 `describe_noise: RELIABILITY: source_identification_accuracy >= 0.80`。但這個 0.80 不是在所有條件下都成立——在安靜環境下準確率可能是 0.95，在極度嘈雜環境下可能降到 0.60。

IR 需要支援「條件可靠性」：

```
PRIM describe_noise(...)
  RELIABILITY:
    WHEN snr > 20dB: accuracy >= 0.95
    WHEN snr 5-20dB: accuracy >= 0.80
    WHEN snr < 5dB: accuracy >= 0.60
    WHEN snr < 0dB: DEGRADED_MODE -> returns only noise_level, not type
```

LLVM IR 不需要這個——因為 ADD 指令在所有條件下都是精確的。但語義指令的可靠性本質上是條件依賴的。

4. 型別系統需要支援「不確定性」

LLVM IR 的型別是精確的——`i32` 就是 32 位整數，不多不少。但語義 IR 的型別需要表達不確定性：

```
TYPE SceneDescription = {
  environment: Enum<indoor|outdoor|vehicle|unknown>,  ; 離散 + unknown
  confidence: float[0,1],                              ; 連續信心度
  alternatives: List<(Enum, float)>                    ; 排名候選列表
  ; 例如：[(indoor, 0.7), (outdoor, 0.2), (vehicle, 0.1)]
}
```

每一個語義型別都應該是一個分布，不只是一個點估計。 這跟傳統型別系統完全不同——傳統型別只關心「這個值是什麼型別」，不關心「這個值有多確定」。

5. ABI（呼叫慣例）需要處理自然語言的模糊性

當 describe_noise 輸出 `{type: "多人交談聲"}` 而 reason_about_scene 需要理解這個輸出時——它依賴的是自然語言的語義，不是精確的符號匹配。如果 describe_noise 這次輸出 `{type: "多人同時說話"}` 而不是 `{type: "多人交談聲"}`——語義一樣但字面不同。

語義 ABI 需要一個「語義等價」的定義——不是字串匹配，而是語義匹配。 這是傳統 ABI 完全不需要面對的問題——因為傳統 ABI 中 `int 42` 就是 `int 42`，沒有模糊空間。

一個可能的解決方案是：在 IR 中強制使用結構化的枚舉型別而非自由文字——把 noise type 從 `string`（任意自然語言）限制為 `Enum<speech|traffic|machine|music|nature|unknown>`。這犧牲了描述的豐富性，但獲得了 ABI 的確定性。**這是語義 IR 設計中一個根本性的 trade-off：描述力 vs. 介面確定性。**

最終的 LLVM ↔ 語義 ISA 對應修正

經過上面的完整展開，我需要修正之前的類比。語義 ISA + Harness 不是 LLVM 的直接翻譯——它有幾個 LLVM 架構中不存在的、全新的結構需求：

LLVM 有的	語義 ISA 也需要的	語義 ISA 額外需要但 LLVM 沒有的
IR	Semantic IR	帶有**條件可靠性**的 IR
Type System	Semantic Type System	帶有**不確定性分布**的型別
Linker	Semantic Linker	基於**語義等價**而非字串匹配的連結
Optimization Passes	GEPA/TextGrad	同時優化確定性和統計性指令的混合優化
Single-pass compilation	—	**控制迴路語義**（輸出影響下一次輸入）
—	—	**多速率執行模式**（快/慢/極慢通道）
Interrupt handling	—	**語義中斷**（偏好變更打斷正常處理流程）
—	—	**向下回饋路徑**（高層語義修改低層物理參數）

這些「額外需要」的部分——控制迴路、多速率執行、語義中斷、向下回饋——每一個都是一個獨立的研究問題。它們不是 LLVM 問題的「語義版本」——它們是語義計算這個新範式**自身產生的**全新問題。

這就是為什麼你的研究不只是「把 LLVM 的概念翻譯到 LLM 時代」——而是在建構一個新的計算基礎設施，它有自己的結構需求，需要自己的理論框架。